

PRODUCT REQUIREMENTS DOCUMENT

Backend Build Plan for Demo

AI Egress Compliance Checker - Demo-Ready Backend Scope, API, and 3-4 Week Build Plan

Version 1.0 - Engineering-facing scope for a working demo
August 2026

1. Purpose of This Document

The full MVP PRD (Egress Checker MVP) scopes what a consultant-usable internal tool needs. This document is narrower: it scopes only what the BACKEND needs to do to support a convincing, working DEMO - to your partner, an early client, or an investor. A demo backend has different priorities than a production MVP backend: speed of build and reliability of the specific demoed path matter more than robustness, edge-case handling, or multi-user support. Build this first; the full MVP backend (Section 10 of this doc) is a superset you grow into afterward, not a separate rebuild.

2. Demo Objective & Audience

Question	Answer
Who is the audience?	Your consultancy partner, and/or a prospective client or informal investor - people who need to believe the core idea works, not production engineers
What's the one thing they must walk away believing?	Upload a real floor plan, and in minutes the system finds real egress violations a human would also flag, with the correct code clause cited
Live demo or recorded?	Plan for live if possible - it's far more convincing - but the plan below includes a recorded/cached fallback (Section 8) in case live processing is flaky on demo day
How long is the demo?	Assume 5-10 minutes of backend-dependent screen time within a larger pitch

3. Demo Script (drives what the backend must support)

- 1. Presenter uploads a pre-selected, real (or realistic) commercial office floor plan (DXF preferred for reliability).
- 2. Backend processes it and returns extracted walls/doors/exits rendered back over the drawing (proves extraction works).
- 3. Backend returns travel-distance and exit-count flags, each showing the violated code clause and measured value vs. limit (proves the compliance logic works).
- 4. Presenter clicks one flag to show it's traceable to a specific, real clause - not a generic "this looks wrong" (proves trustworthiness, the hardest thing to sell in this category).
- 5. Presenter marks a flag reviewed/confirmed and exports a one-page summary (proves the workflow, not just the model, is real).

Everything the backend needs to build maps directly to these five steps - nothing else is required for demo day.

4. Backend Scope for the Demo

4.1 In scope

- Single-file upload endpoint (DXF primary path; PDF nice-to-have if time allows, since DXF is materially more reliable to demo).
- Synchronous or near-synchronous processing for one file at a time - no need for a distributed job queue at demo scale.
- DXF parsing to extract walls, doors, rooms, and exit candidates.
- Walkable path graph construction and travel-distance calculation to nearest exit.

- A small, hand-curated table of the specific UAE Fire and Life Safety Code clauses needed for travel distance and exit count (10-20 clauses is enough for a demo building type).
- Violation detection logic comparing measured values to code limits, returning clause reference + measured value + limit.
- Endpoints to serve extracted geometry and flags to the frontend for overlay rendering.
- A basic "mark reviewed" endpoint and a summary export endpoint (PDF or CSV).
- One seeded demo project with 1-2 known-good floor plans, pre-tested end to end before demo day.

4.2 Explicitly out of scope for the demo backend

- Authentication/multi-user accounts (a single hardcoded demo user or no auth at all is fine).
- Multi-tenant isolation, billing, usage metering.
- Occupant-load edge cases beyond the demo building's occupancy type.
- PDF/CV fallback robustness on arbitrary real-world scanned drawings - only needs to work on the specific pre-tested demo file(s).
- Production-grade error handling for every malformed input - just don't crash on the one file you're demoing.
- LLM/RAG explanation layer - deterministic clause-table lookups are sufficient and arguably more convincing for a demo (no hallucination risk).
- Horizontal scaling, load testing, uptime SLAs.

5. API Design (demo scope)

Method & path	Purpose	Notes
POST /projects	Create a demo project	Minimal fields: name, client_name
POST /projects/{id}/drawings	Upload a DXF (or PDF) file and kick off processing	Returns a drawing_id and a status (processing/ready/failed)
GET /drawings/{id}/status	Poll processing status	Simple polling is fine at demo scale - no websockets needed
GET /drawings/{id}/elements	Return extracted geometry (walls, doors, rooms, exits) as GeoJSON-like coordinates	Frontend uses this to render the overlay
GET /drawings/{id}/violations	Return the list of flags: type, clause_ref, measured_value, limit_value, severity, related geometry	The core "payoff" endpoint of the demo
PATCH /violations/{id}	Update a flag's status (confirmed/false_positive/resolved) and optional note	Powers the "presenter marks it reviewed" beat
GET /drawings/{id}/export	Generate and return a PDF or CSV summary	Can be a simple templated PDF for demo purposes

6. Data Model (demo-minimal)

Entity	Key fields
Project	id, name, client_name

Entity	Key fields
Drawing	id, project_id, file_url, file_type, occupancy_type, scale, status
ExtractedElement	id, drawing_id, type (wall/door/exit/room), geometry
Violation	id, drawing_id, type, related_element_id, clause_ref, measured_value, limit_value, severity, status, note
CodeClause (static reference table)	id, code_name, clause_number, description, applies_to (occupancy_type), limit_type, limit_value

This is intentionally a subset of the full MVP data model - Room and ExitPath and Review as separate entities can be collapsed into Drawing/Violation fields for the demo, then split back out when you build toward the full MVP.

7. Processing Pipeline (what actually runs on upload)

Step	What happens	Library
1. Parse	Read DXF layers, extract line/polyline entities for walls, block references for doors	ezdxf
2. Classify	Heuristically tag doors as interior vs. exit based on layer name, block type, or proximity to building perimeter; allow a manual override list for the demo file specifically	Custom logic + ezdxf
3. Build graph	Convert rooms/corridors into nodes, connections into edges	NetworkX
4. Calculate	Shortest path from each room centroid to nearest exit node; sum edge lengths using drawing scale	NetworkX + Shapely
5. Check	Compare each calculated distance and each floor's exit count/width to the CodeClause table for the tagged occupancy type	Custom rules logic (plain Python, no ML)
6. Store	Persist ExtractedElement and Violation rows	PostgreSQL
7. Serve	Expose via the Section 5 API for the frontend to render	FastAPI

8. Demo-Day Reliability Plan

- Pre-test the exact demo file(s) at least 3 times end-to-end in the days before the demo, on the same environment you'll present from.
- Cache the fully-processed result for the demo file server-side, so if live processing is slow or flaky on the day, the app can serve the pre-computed result instantly while still looking live.
- Have a short backup screen-recording of the full flow as an absolute fallback if live demo access (wifi, laptop, etc.) fails - not to replace the live demo, just as insurance.
- Keep the demo file's building geometry moderately simple (single floor, clear room boundaries) - save visually complex, edge-case-heavy drawings for after the demo.

9. Build Plan - 3 to 4 Weeks

Week	Focus	Exit criteria
1	FastAPI project scaffold, Postgres schema (Section 6), file upload endpoint, DXF parsing for walls/doors/rooms	Upload a DXF and get back extracted elements as JSON
2	Exit classification + path graph + travel-distance calculation; build the CodeClause table (10-20 clauses) and wire in violation detection	GET /drawings/{id}/violations returns correct, clause-cited flags on the test file
3	Wire endpoints to a minimal frontend overlay (can be rough - this doc is backend-focused, but the demo needs something clickable); PATCH status endpoint; export endpoint	Full demo script (Section 3) runs end to end on the test file
4 (buffer)	Reliability pass: re-run on the actual demo file(s) repeatedly, add the caching fallback (Section 8), fix anything broken by real-world DXF quirks	Demo file processes correctly and fast, 3 clean runs in a row, on the actual presentation setup

10. How This Grows Into the Full MVP Backend

Nothing here is throwaway. The demo backend's schema, API shape, and pipeline are a strict subset of the full Egress Checker MVP PRD - you're building the real foundation, just without auth, multi-file robustness, and the PDF/CV fallback hardening. After the demo, the next work is: add authentication and multi-project support, harden the PDF/CV extraction path for arbitrary real drawings, expand the CodeClause table to full travel-distance and exit-count coverage per the MVP PRD, and add the async job queue once you're processing more than one file at a time. None of that requires re-architecting what you build for the demo.

11. Tech Stack Recap (demo-scoped)

Layer	Choice	Why this is enough for a demo
Backend framework	Python FastAPI	Fast to build, good for the ML/geometry-heavy logic, easy to demo via simple HTTP calls
CAD parsing	ezdxf	Reliable, deterministic path - avoid the CV fallback for the demo file itself
Graph/geometry	NetworkX + Shapely	Deterministic distance calculations - the most trustworthy part of the demo
Database	PostgreSQL (a single local or small hosted instance is fine)	No need for PostGIS optimization or scaling at demo volume
File storage	Local disk or a single S3 bucket	No need for multi-tenant encryption/isolation yet - just don't lose the demo file
Processing	Synchronous request or a simple background task (FastAPI BackgroundTasks) - no Celery/Redis needed yet	One file at a time, presenter-triggered - a queue adds complexity you don't need for demo day
Frontend (minimal)	A simple React page consuming the API, canvas overlay for flags	Doesn't need to be polished - it needs to make the backend's output visible and clickable